Binary_Classification_Bank.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Share

Files

..

sample_data

Churn_Modelling.csv

churn_model.h5

Disk 80.01 GB available

+ Code + Text

T4 RAM Disk Gemini

We need to train a model that predicts if a customer will leave the bank or not.

```
[2] import pandas as pd  # good ML habits
```

```
[ ] from google.colab import drive
    drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
[3] dataset = pd.read_csv('/content/Churn_Modelling.csv')
    #Kaggle database (source: https://www.kaggle.com/aakash50897/churn-modellingcsv?select=Churn_Modelling.csv)
    dataset
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exited
0	1	15634602	Hargrave	619	France	Female	42	2	0.00	1	1	1	101348.88	1
1	2	15647311	Hill	608	Spain	Female	41	1	83807.86	1	0	1	112542.58	0
2	3	15619304	Onio	502	France	Female	42	8	159660.80	3	1	0	113931.57	1
3	4	15701354	Boni	699	France	Female	39	1	0.00	2	0	0	93826.63	0
4	5	15737888	Mitchell	850	Spain	Female	43	2	125510.82	1	1	1	79084.10	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
9995	9996	15606229	Obijiaku	771	France	Male	39	5	0.00	2	1	0	96270.64	0
9996	9997	15569892	Johnstone	516	France	Male	35	10	57369.61	1	1	1	101699.77	0
9997	9998	15584532	Liu	709	France	Female	36	7	0.00	1	0	1	42085.58	1
9998	9999	15682355	Sabbatini	772	Germany	Male	42	3	75075.31	2	1	0	92888.52	1
9999	10000	15628319	Walker	792	France	Female	28	4	130142.79	1	1	0	38190.78	0

10000 rows × 14 columns

Next steps: Generate code with dataset View recommended plots New interactive sheet

```
[ ] print(dataset.columns)
```

```
Index(['RowNumber', 'CustomerId', 'Surname', 'CreditScore', 'Geography',
       'Gender', 'Age', 'Tenure', 'Balance', 'NumOfProducts', 'HasCrCard',
       'IsActiveMember', 'EstimatedSalary', 'Exited'],
      dtype='object')
```

```
[ ] print(dataset.index)
```

```
RangeIndex(start=0, stop=10000, step=1)
```

```
[9] #Not all independent variables are important for the result (such as RowNumber, CustomerId)
    X = dataset.iloc[:, 3: 13].values
    X
```

```
array([[619, 'France', 'Female', ..., 1, 1, 101348.88],
       [608, 'Spain', 'Female', ..., 0, 1, 112542.58],
       [502, 'France', 'Female', ..., 1, 0, 113931.57],
       ...,
       [709, 'France', 'Female', ..., 0, 1, 42085.58],
       [772, 'Germany', 'Male', ..., 1, 0, 92888.52],
       [792, 'France', 'Female', ..., 1, 0, 38190.78]], dtype=object)
```

```
[10] #labels
     y = dataset.iloc[:, 13].values
     y
```

```
array([1, 0, 1, ..., 1, 1, 0])
```

```
[ ] #Data encoding :
    #We have to encode categorical data (such as Geography and Gender)
    #ORDINAL ENCODING - way 1:
    from sklearn.preprocessing import LabelEncoder  # one-hot encoder

    X_ord_1 = dataset.iloc[:, 3: 13].values

    labelencoder_X = LabelEncoder() #instantiate an object of the class LabelEncoder
    X_ord_1[:, 1] = labelencoder_X.fit_transform(X_ord_1[:, 1]) #ordinal encoding for column 1

    X_ord_1[:, 2] = labelencoder_X.fit_transform(X_ord_1[:, 2]) #ordinal encoding for column 2

    X_ord_1
```

```
array([[619, 0, 0, ..., 1, 1, 101348.88],
       [608, 2, 0, ..., 0, 1, 112542.58],
       [502, 0, 0, ..., 1, 0, 113931.57],
       ...,
       [709, 0, 0, ..., 0, 1, 42085.58],
       [772, 1, 1, ..., 1, 0, 92888.52],
       [792, 0, 0, ..., 1, 0, 38190.78]], dtype=object)
```

```
[ ] #ORDINAL ENCODING - way 2:  #bug - sets all values to zero!
    from sklearn.preprocessing import OrdinalEncoder

    X_ord_2 = dataset.iloc[:, 3: 13].values

    ordinal_encoder_1 = OrdinalEncoder()
    X_ord_2[:, 1] = ordinal_encoder_1.fit_transform([X_ord_2[:, 1]])
    X_ord_2[:, 2] = ordinal_encoder_1.fit_transform([X_ord_2[:, 2]])

    X_ord_2
```

```
array([[619, 0.0, 0.0, ..., 1, 1, 101348.88],
       [608, 0.0, 0.0, ..., 0, 1, 112542.58],
       [502, 0.0, 0.0, ..., 1, 0, 113931.57],
       ...,
       [709, 0.0, 0.0, ..., 0, 1, 42085.58],
       [772, 0.0, 0.0, ..., 1, 0, 92888.52],
       [792, 0.0, 0.0, ..., 1, 0, 38190.78]], dtype=object)
```

```
[ ] X = X_ord_1
```

```
[ ] #ONE-HOT ENCODING :
    #Way 1 : using data values :
    from sklearn.preprocessing import OneHotEncoder
    from sklearn.compose import ColumnTransformer
    import numpy as np

    ct = ColumnTransformer( #'encoder' is the name of the column transformer
        [('encoder', OneHotEncoder(), [1,2])],    # The column numbers to be transformed (here is [1] but can be [0, 1, 3])
        remainder='passthrough'                         # Leave the rest of the columns untouched
    )
```

```
X = np.array(ct.fit_transform(X)) #Note: The X matrix should be ordinally encoded (with ordinal encoding applied to it)
df = pd.DataFrame(X)
df
```

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
0	1.0	0.0	1.0	0.0	1.0	619	Female	42	2	0.0	1	1	1	101348.88
1	0.0	1.0	1.0	0.0	0.0	608	Female	41	1	83807.86	1	0	1	112542.58
2	1.0	0.0	1.0	0.0	1.0	502	Female	42	8	159660.8	3	1	0	113931.57
3	1.0	0.0	1.0	0.0	1.0	699	Female	39	1	0.0	2	0	0	93826.63
4	0.0	1.0	1.0	0.0	0.0	850	Female	43	2	125510.82	1	1	1	79084.1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
9995	1.0	0.0	1.0	0.0	1.0	771	Male	39	5	0.0	2	1	0	96270.64
9996	1.0	0.0	1.0	0.0	1.0	516	Male	35	10	57369.61	1	1	1	101699.77
9997	1.0	0.0	1.0	0.0	1.0	709	Female	36	7	0.0	1	0	1	42085.58
9998	1.0	0.0	0.0	1.0	1.0	772	Male	42	3	75075.31	2	1	0	92888.52
9999	1.0	0.0	1.0	0.0	1.0	792	Female	28	4	130142.79	1	1	0	38190.78

10000 rows × 14 columns

```
[ ] #We remove the first column to avoid the dummy data trap
'''Dummy data trap : A scenario where independent variables are highly correlated (one variable predicts the value of others).
In one-hot encoding, one dummy variable can be predicted through other dummy variables, thus causing redundancy
==> Using all dummy variables for regression models leads to dummy variable trap
==> We exclude one of those dummy variables.'''

X = X[:, 1:]
df = pd.DataFrame(X)
df
```

	0	1	2	3	4	5	6	7	8	9	10
0	0.0	0.0	619	Female	42	2	0.0	1	1	1	101348.88
1	0.0	1.0	608	Female	41	1	83807.86	1	0	1	112542.58
2	0.0	0.0	502	Female	42	8	159660.8	3	1	0	113931.57
3	0.0	0.0	699	Female	39	1	0.0	2	0	0	93826.63
4	0.0	1.0	850	Female	43	2	125510.82	1	1	1	79084.1
...	...	...	...	...	...	...	...	...	...	...	...
9995	0.0	0.0	771	Male	39	5	0.0	2	1	0	96270.64
9996	0.0	0.0	516	Male	35	10	57369.61	1	1	1	101699.77
9997	0.0	0.0	709	Female	36	7	0.0	1	0	1	42085.58
9998	1.0	0.0	772	Male	42	3	75075.31	2	1	0	92888.52
9999	0.0	0.0	792	Female	28	4	130142.79	1	1	0	38190.78

10000 rows × 11 columns

```
[11] #ONE-HOT ENCODING : Way 2 : using data frame :
X_df = dataset.iloc[:, 3: 13]
X_df = pd.concat([X_df, pd.get_dummies(X_df['Gender'], drop_first=True)], axis=1)  #drops the first column
#axis = 1 means to concatenate along the columns (put one column beside another)
X_df.drop(['Gender'], axis=1, inplace=True)  #get rid of the original Geography column
X_df
```

	CreditScore	Geography	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Male
0	619	France	42	2	0.00	1	1	1	101348.88	False
1	608	Spain	41	1	83807.86	1	0	1	112542.58	False
2	502	France	42	8	159660.80	3	1	0	113931.57	False
3	699	France	39	1	0.00	2	0	0	93826.63	False
4	850	Spain	43	2	125510.82	1	1	1	79084.10	False
...	...	...	...	...	...	...	...	...	...	...
9995	771	France	39	5	0.00	2	1	0	96270.64	True
9996	516	France	35	10	57369.61	1	1	1	101699.77	True
9997	709	France	36	7	0.00	1	0	1	42085.58	False
9998	772	Germany	42	3	75075.31	2	1	0	92888.52	True
9999	792	France	28	4	130142.79	1	1	0	38190.78	False

10000 rows × 10 columns

Next steps: Generate code with X_df | View recommended plots | New interactive sheet

```
[12] X_df = pd.concat([X_df, pd.get_dummies(X_df['Geography'], prefix='country', drop_first=True)], axis=1)  #drops the first column
#axis = 1 means to concatenate along the columns (put one column beside another)
X_df.drop(['Geography'], axis=1, inplace=True)  #get rid of the original Geography column
X_df
```

	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Male	country_Germany	country_Spain
0	619	42	2	0.00	1	1	1	101348.88	False	False	False
1	608	41	1	83807.86	1	0	1	112542.58	False	False	True
2	502	42	8	159660.80	3	1	0	113931.57	False	False	False
3	699	39	1	0.00	2	0	0	93826.63	False	False	False
4	850	43	2	125510.82	1	1	1	79084.10	False	False	True
...	...	...	...	...	...	...	...	...	...	...	...
9995	771	39	5	0.00	2	1	0	96270.64	True	False	False
9996	516	35	10	57369.61	1	1	1	101699.77	True	False	False
9997	709	36	7	0.00	1	0	1	42085.58	False	False	False
9998	772	42	3	75075.31	2	1	0	92888.52	True	True	False
9999	792	28	4	130142.79	1	1	0	38190.78	False	False	False

10000 rows × 11 columns

Next steps: Generate code with X_df | View recommended plots | New interactive sheet

```
[ ] X_df = pd.concat([X_df, pd.get_dummies(X_df['Gender'], prefix='gender', drop_first=True)], axis=1)  #drops the first column
#axis = 1 means to concatenate along the columns (put one column beside another)
X_df.drop(['Gender'], axis=1, inplace=True)  #get rid of the original Geography column
X_df
```

	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	country_Germany	country_Spain	gender_Male
0	619	42	2	0.00	1	1	1	101348.88	False	False	False
1	608	41	1	83807.86	1	0	1	112542.58	False	True	False

2	502	42	8	159660.80	3	1	0	113931.57	False	False	False
3	699	39	1	0.00	2	0	0	93826.63	False	False	False
4	850	43	2	125510.82	1	1	1	79084.10	False	True	False
...	...	...	...	...	...	...	...	...	...	...	...
9995	771	39	5	0.00	2	1	0	96270.64	False	False	True
9996	516	35	10	57369.61	1	1	1	101699.77	False	False	True
9997	709	36	7	0.00	1	0	1	42085.58	False	False	False
9998	772	42	3	75075.31	2	1	0	92888.52	True	False	True
9999	792	28	4	130142.79	1	1	0	38190.78	False	False	False

10000 rows × 11 columns

[13]
```
# re_split dataframe into X and y
X = X_df.values
```

[14]
```
# Split the data into training and test set (20% for the test set)

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,test_size = 0.2, random_state = 1) # We use random_state to make sure splitting contains the same data each time.
```

[15]
```
#Standardise the data (x_standardised = (x - x_mean)/std_dev)
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test) #note that we use the scale set from the training set to transform the test set
df = pd.DataFrame(X_train)
df
```

	0	1	2	3	4	5	6	7	8	9	10
0	-0.230820	-0.944500	-0.701742	0.588173	0.802257	-1.553374	0.977259	0.427394	0.915091	1.714901	-0.572731
1	-0.251509	-0.944500	-0.355203	0.469849	0.802257	-1.553374	-1.023271	-1.025487	-1.092788	-0.583124	-0.572731
2	-0.396330	0.774987	0.337876	0.858788	-0.911510	0.643760	0.977259	-0.944798	-1.092788	1.714901	-0.572731
3	-0.044622	1.252622	0.337876	0.565604	0.802257	-1.553374	0.977259	-0.551946	-1.092788	1.714901	-0.572731
4	0.658795	-0.562392	1.030954	0.730395	-0.911510	-1.553374	-1.023271	1.083383	0.915091	-0.583124	1.746019
...	...	...	...	...	...	...	...	...	...	...	...
7995	-0.303231	0.774987	0.684415	0.495439	-0.911510	0.643760	0.977259	-0.579180	0.915091	1.714901	-0.572731
7996	0.348464	2.303420	-0.701742	0.076679	-0.911510	0.643760	-1.023271	-0.529780	-1.092788	1.714901	-0.572731
7997	0.224332	0.583933	1.377494	-1.225992	-0.911510	0.643760	0.977259	-0.140969	-1.092788	-0.583124	-0.572731
7998	0.131233	0.010771	1.030954	-1.225992	0.802257	0.643760	0.977259	0.017812	-1.092788	-0.583124	-0.572731
7999	1.165670	0.297352	0.337876	0.379955	-0.911510	0.643760	-1.023271	-1.158225	0.915091	1.714901	-0.572731

8000 rows × 11 columns

Next steps: Generate code with df | View recommended plots | New interactive sheet

[16]
```
#Building the model :
#We use 2 dense layers

import tensorflow as tf
from tensorflow import keras  # tensorflow api

model = keras.Sequential()

#add first hidden layer
model.add(tf.keras.layers.Dense(units=6, kernel_initializer='uniform', activation='relu'))

#add 2nd hidden layer
model.add(tf.keras.layers.Dense(units=6, kernel_initializer='uniform', activation='relu'))

'''The number of units is mainly chosen by experience. In general, based on experimentation,
setting it to the average between the number of the input nodes (11) and the number of the
ouput nodes (1). Cross-validation can also be used to choose to choose the best parameters (parameter tuning).'''

'''random_uniform: Weights are initialized to uniformly random small values between -0.05 to 0.05.
random_normal: Weights are initialized according to a Gaussian distribution, with zero mean and a small standard deviation of 0.05.
zero: All weights are initialized to zero.'''
```

```
'random_uniform: Weights are initialized to uniformly random small values between -0.05 to 0.05.\nrandom_normal: Weights are initialized according to a Gaussian distribution, with zero mean and a small standard deviation of 0.05.\nzero: All weights are initialized to zero.'
```

[17]
```
#Add the output layer
model.add(keras.layers.Dense(units=1, kernel_initializer='uniform', activation='sigmoid')) #Sigmoid for binary, Softmax for multiclass
```

[18]
```
#Compilation
model.compile(optimizer = 'adam', loss ='binary_crossentropy', metrics = ['accuracy'])
```

[ ]
```
'''
Note: If you want to personalize an optimizer, do the following:
opt = keras.optimizers.Adam(learning_rate=0.01)
model.compile(loss='categorical_crossentropy', optimizer=opt)
'''

'''
Here are the parameters of the Adam optimizer:
tf.keras.optimizers.Adam(
    learning_rate=0.001,
    beta_1=0.9,
    beta_2=0.999,
    epsilon=1e-07,
    amsgrad=False,
    name="Adam",
    **kwargs
)
'''
```

[ ]
```
'''
Note: You can use a learning rate schedule to modulate how the learning rate of your optimizer changes over time
lr_schedule = keras.optimizers.schedules.ExponentialDecay(
    initial_learning_rate=1e-2,
    decay_steps=10000,
    decay_rate=0.9)
optimizer = keras.optimizers.SGD(learning_rate=lr_schedule)
'''
```

[19]
```
#Training
history = model.fit(X_train, y_train, batch_size = 10, epochs = 25, verbose=2)  # train the model
```

```
Epoch 1/25
800/800 - 4s - 5ms/step - accuracy: 0.7968 - loss: 0.4808
Epoch 2/25
800/800 - 4s - 5ms/step - accuracy: 0.7972 - loss: 0.4278
Epoch 3/25
800/800 - 2s - 3ms/step - accuracy: 0.8006 - loss: 0.4228
Epoch 4/25
```

```
800/800 - 1s - 1ms/step - accuracy: 0.8249 - loss: 0.4188
Epoch 5/25
800/800 - 1s - 1ms/step - accuracy: 0.8280 - loss: 0.4165
Epoch 6/25
800/800 - 1s - 2ms/step - accuracy: 0.8304 - loss: 0.4152
Epoch 7/25
800/800 - 1s - 1ms/step - accuracy: 0.8313 - loss: 0.4130
Epoch 8/25
800/800 - 1s - 1ms/step - accuracy: 0.8322 - loss: 0.4122
Epoch 9/25
800/800 - 1s - 1ms/step - accuracy: 0.8334 - loss: 0.4107
Epoch 10/25
800/800 - 2s - 2ms/step - accuracy: 0.8340 - loss: 0.4099
Epoch 11/25
800/800 - 4s - 4ms/step - accuracy: 0.8350 - loss: 0.4083
Epoch 12/25
800/800 - 2s - 3ms/step - accuracy: 0.8351 - loss: 0.4078
Epoch 13/25
800/800 - 2s - 2ms/step - accuracy: 0.8334 - loss: 0.4071
Epoch 14/25
800/800 - 1s - 1ms/step - accuracy: 0.8350 - loss: 0.4065
Epoch 15/25
800/800 - 1s - 1ms/step - accuracy: 0.8344 - loss: 0.4059
Epoch 16/25
800/800 - 1s - 2ms/step - accuracy: 0.8351 - loss: 0.4056
Epoch 17/25
800/800 - 2s - 2ms/step - accuracy: 0.8344 - loss: 0.4053
Epoch 18/25
800/800 - 1s - 2ms/step - accuracy: 0.8351 - loss: 0.4044
Epoch 19/25
800/800 - 1s - 1ms/step - accuracy: 0.8346 - loss: 0.4043
Epoch 20/25
800/800 - 1s - 2ms/step - accuracy: 0.8356 - loss: 0.4042
Epoch 21/25
800/800 - 1s - 1ms/step - accuracy: 0.8357 - loss: 0.4037
Epoch 22/25
800/800 - 1s - 1ms/step - accuracy: 0.8340 - loss: 0.4036
Epoch 23/25
800/800 - 1s - 1ms/step - accuracy: 0.8346 - loss: 0.4030
Epoch 24/25
800/800 - 1s - 1ms/step - accuracy: 0.8350 - loss: 0.4034
Epoch 25/25
800/800 - 1s - 2ms/step - accuracy: 0.8349 - loss: 0.4030
```

[20]
```
loss, accuracy = model.evaluate(X_test, y_test)
```

```
63/63 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8350 - loss: 0.4017
```

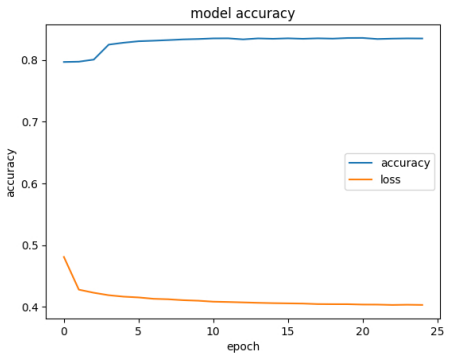
[21]
```
#Evaluation
y_pred = model.predict(X_test)
print("PREDICTIONS:",y_pred)
y_pred = (y_pred > 0.5)
print(y_pred)
```

```
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step
PREDICTIONS: [[0.16335618]
 [0.09069093]
 [0.21373712]
 ...
 [0.05228872]
 [0.10618393]
 [0.33672497]]
[[False]
 [False]
 [False]
 ...
 [False]
 [False]
 [False]]
```

[22]
```
#Predict using the info of a new customer
new_customer = [[0, 0, 600, 1, 40, 3, 60000, 2, 1, 1, 50000]]
new_customer = sc.transform(new_customer)
new_prediction = model.predict(new_customer)
new_prediction = (new_prediction > 0.5)
print(new_prediction)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 121ms/step
[[False]]
```

[23]
```
from matplotlib import pyplot as plt
plt.plot(history.history['accuracy'])
plt.plot(history.history['loss'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['accuracy','loss'])
plt.show()
```

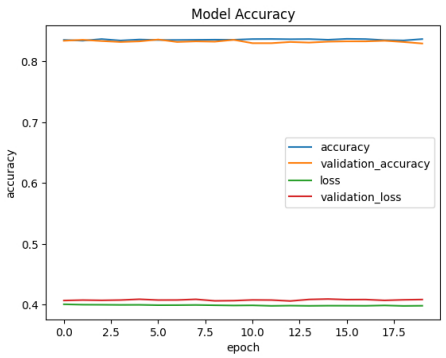


[25]
```
#Note : An alternative to using train_test_split() is to specify a validation_split percentage.
#This is done when fitting the model, for example:

from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X = sc.fit_transform(X)

history = model.fit(X,y, verbose = 1,
                    validation_split = 0.2, # split data in 80-20 sets
                    epochs = 20,
                    batch_size = 10)
```

```
Epoch 1/20
800/800 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.8408 - loss: 0.3935 - val_accuracy: 0.8340 - val_loss: 0.4068
Epoch 2/20
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8280 - loss: 0.4090 - val_accuracy: 0.8355 - val_loss: 0.4075
Epoch 3/20
800/800 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.8393 - loss: 0.3986 - val_accuracy: 0.8335 - val_loss: 0.4071
Epoch 4/20
800/800 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.8356 - loss: 0.4028 - val_accuracy: 0.8320 - val_loss: 0.4076
Epoch 5/20
800/800 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.8423 - loss: 0.3931 - val_accuracy: 0.8330 - val_loss: 0.4089
```

```
Epoch 6/20
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8365 - loss: 0.3948 - val_accuracy: 0.8360 - val_loss: 0.4076
Epoch 7/20
800/800 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.8286 - loss: 0.4098 - val_accuracy: 0.8320 - val_loss: 0.4077
Epoch 8/20
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8365 - loss: 0.3982 - val_accuracy: 0.8330 - val_loss: 0.4088
Epoch 9/20
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8308 - loss: 0.4064 - val_accuracy: 0.8325 - val_loss: 0.4063
Epoch 10/20
800/800 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.8292 - loss: 0.4061 - val_accuracy: 0.8355 - val_loss: 0.4066
Epoch 11/20
800/800 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.8402 - loss: 0.3943 - val_accuracy: 0.8300 - val_loss: 0.4078
Epoch 12/20
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8462 - loss: 0.3849 - val_accuracy: 0.8300 - val_loss: 0.4076
Epoch 13/20
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8371 - loss: 0.3967 - val_accuracy: 0.8320 - val_loss: 0.4060
Epoch 14/20
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8343 - loss: 0.4078 - val_accuracy: 0.8310 - val_loss: 0.4086
Epoch 15/20
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8313 - loss: 0.4012 - val_accuracy: 0.8325 - val_loss: 0.4093
Epoch 16/20
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8310 - loss: 0.4068 - val_accuracy: 0.8330 - val_loss: 0.4083
Epoch 17/20
800/800 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.8430 - loss: 0.3933 - val_accuracy: 0.8330 - val_loss: 0.4084
Epoch 18/20
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8342 - loss: 0.3962 - val_accuracy: 0.8340 - val_loss: 0.4070
Epoch 19/20
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8341 - loss: 0.4003 - val_accuracy: 0.8320 - val_loss: 0.4080
Epoch 20/20
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8353 - loss: 0.3960 - val_accuracy: 0.8295 - val_loss: 0.4084
```

[26]
```python
from matplotlib import pyplot as plt
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.legend(['accuracy','validation_accuracy', 'loss', 'validation_loss'])
plt.title('Model Accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.show()
```



[27]
```python
model.save('/content/churn_model.h5')
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the native Keras fo
```

[28]
```python
import tensorflow as tf
from tensorflow import keras  # tensorflow api

my_trained_model = keras.models.load_model('/content/churn_model.h5')
print(my_trained_model)
```

```
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile_metrics` will be empty until you train or evaluate the model.
<Sequential name=sequential, built=True>
```

[30]
```python
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()

#Predict using the info of a new customer
new_customer = [[0, 0, 600, 1, 40, 3, 60000, 2, 1, 1, 50000]]
new_customer = sc.fit_transform(new_customer)
new_prediction = my_trained_model.predict(new_customer)
new_prediction = (new_prediction > 0.5)
print(new_prediction)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 181ms/step
[[False]]
```